

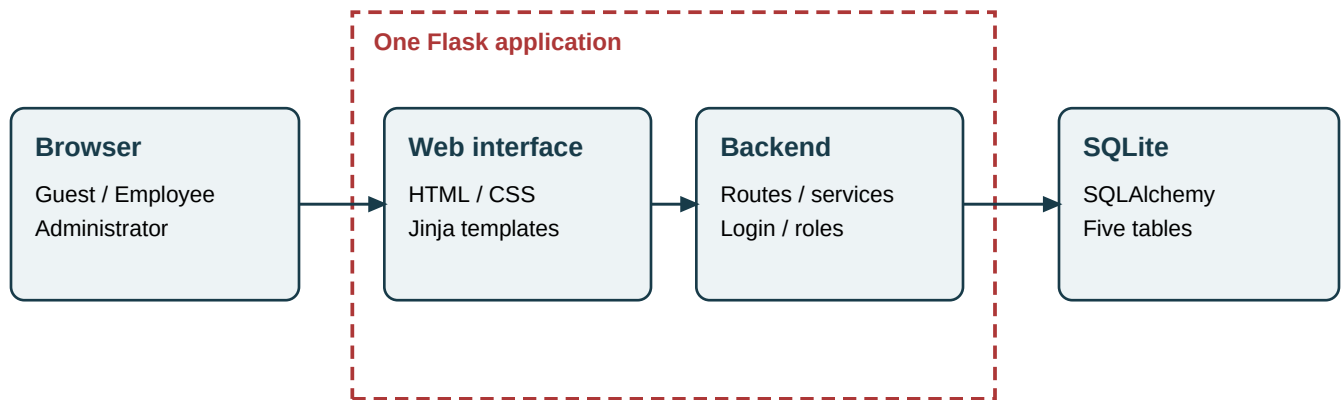
Secure Art Gallery Web Application

Secure Design Document

BIBIFI Group 11 | Huseyin Simsek and Gustavo Pepe | October 3, 2026

Architecture and scope

This design extends Gustavo Pepe's BIBIFI doc.docx. It retains the browser-to-Flask flow, authentication, event processing, history and occupancy queries, and relational stores. The changes make trust boundaries explicit, complete STRIDE, correct movement rules, and distinguish implemented controls from planned deployment work.



HTTP on loopback now; HTTPS planned for network deployment.

Responses return encoded HTML; data persists in instance/gallery.db.

The web application and backend are logical layers within one Flask process. Jinja renders the interface; route handlers authenticate and authorize; service functions validate movements; SQLAlchemy accesses SQLite. This meets Browser -> Web Application -> Backend/API -> Database without a separate REST server.

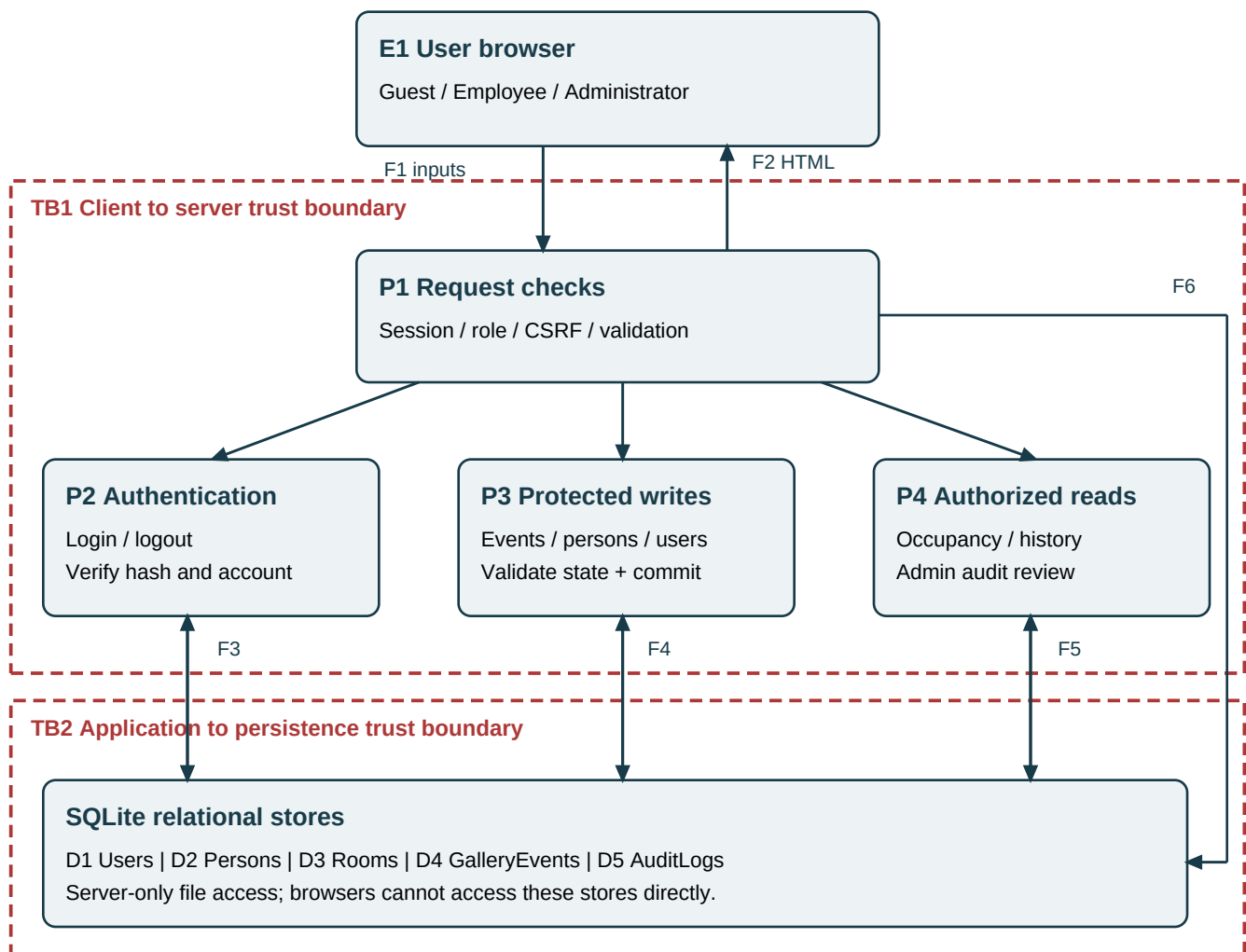
Trust assumptions

Treat all browser fields and cookies as untrusted until validated. A signed session identifies an account but does not grant a role by itself: active status, session version and role come from Users. The application host and database file owner are trusted administrators; the application cannot prevent them from modifying files directly.

Simplicity decisions

Guest accounts see aggregate occupancy only, removing the need for a guest-to-person ownership mapping. No separate Sessions or PersonState table is required. State is reconstructed from the append-only event ledger. Normal staff actions have role decorators; write transactions recheck the actor. Database audit triggers and a redundant default-deny hook are not used; explicit handlers and route tests make enforcement easier to explain.

Data flow and trust boundaries



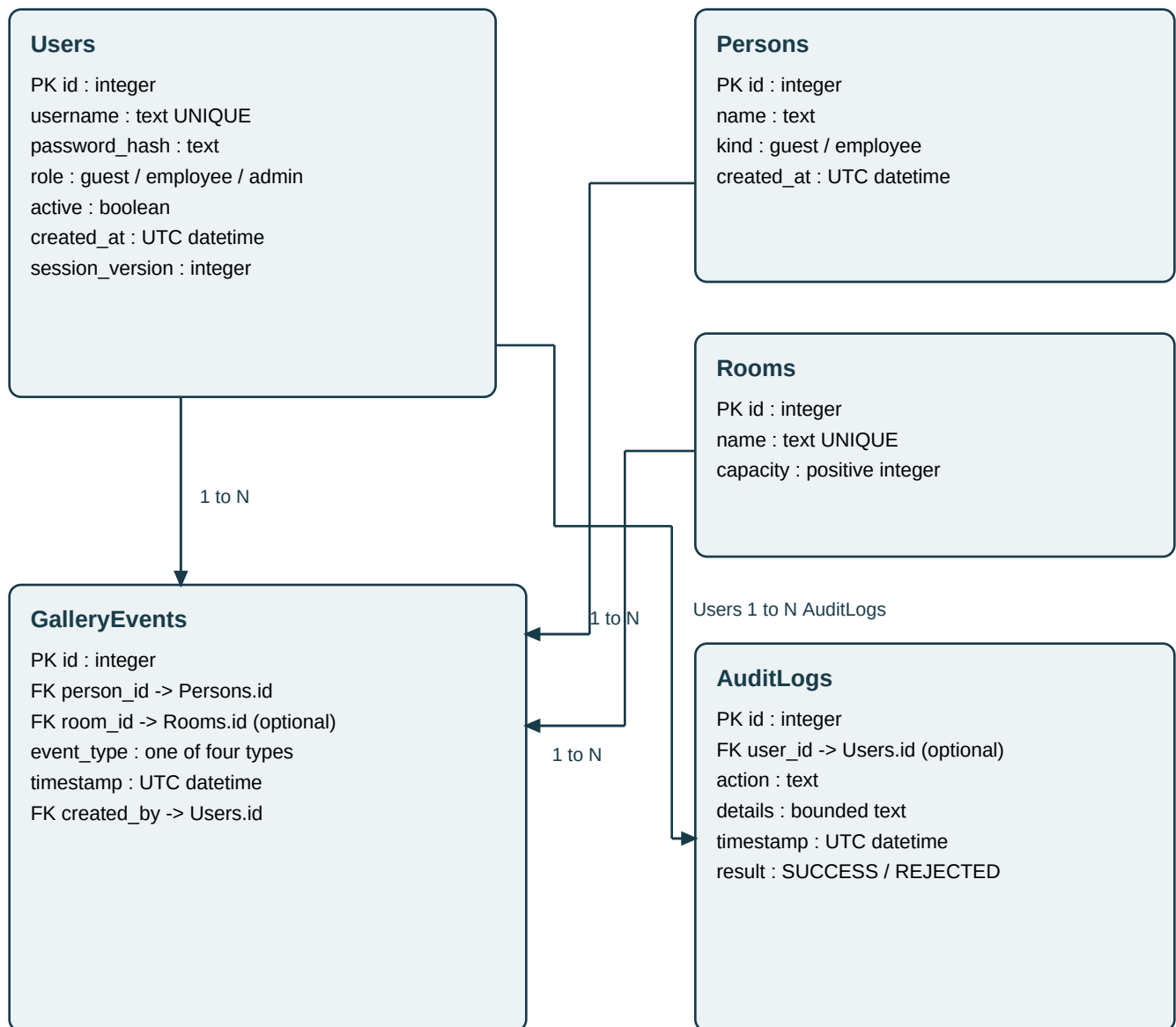
External entity E1 is the browser operated by a Guest, Employee or Administrator. Process P1 validates the session, CSRF token, role and input. P2 authenticates or logs out; P3 manages persons, movements and users; P4 queries occupancy, history and audit records. D1-D5 are the five SQLite stores.

TB1 separates untrusted client input from trusted server decisions. Flow F1 carries credentials, signed cookies and forms to P1; F2 returns encoded HTML and a session cookie. TLS across TB1 is PLANNED for network deployment; the local demo is HTTP on loopback.

TB2 separates application authorization from persistence. F3 reads password hashes and account state; F4 reads reference data and committed history, writes validated events, and changes authorized user fields; F5 returns authorized results; F6 records security actions. Database writes use server credentials/file access, never browser access. SQLAlchemy binds values. SQLite checks foreign keys and schema constraints.

Dashed red rectangles are trust boundaries. Arrows represent the labeled request/data exchanges; return flows include query results and status. The local .env stays outside the stores and source control and supplies the signing secret only to the server.

Relational database design



Users 1:N GalleryEvents through created_by; Persons 1:N GalleryEvents through person_id; Rooms 1:N GalleryEvents through optional room_id; Users 1:N AuditLogs through optional user_id. Unknown login attempts have no identified audit user. Person kind distinguishes tracked guests and employees; it does not grant application permissions.

All IDs are primary keys. Usernames and room names are unique. CHECK constraints limit roles, person kinds, event types, positive capacities, audit outcomes and event/room nullability. Foreign keys are enabled for every connection. Event person/time and audit action/time fields are indexed. schema.sql is generated from these SQLAlchemy models and tested by loading it into an empty SQLite database.

No stored plaintext password, session token or secret is part of the schema. Creation/event/audit times use UTC. The ledger ID defines append order. Event timestamps are server-generated and at least one microsecond later than the previous timestamp if the server clock moves backward.

Movement and transaction design

Current state	Accepted event	New state
Outside	ENTER_GALLERY with no room	In gallery lobby
In gallery lobby	ENTER_ROOM for an existing room with capacity	In that room
In room R	LEAVE_ROOM naming R	In gallery lobby
In gallery lobby	LEAVE_GALLERY with no room	Outside

All other transitions are rejected. No room exit from outside, no duplicate entry, no second simultaneous room, no wrong-room exit, and no gallery exit from a room are allowed. Staff cannot choose an actor or timestamp. An invalid person/room reference also fails.

Serialized append procedure

1. Authenticate the session and check the staff role; validate form fields and the CSRF token.
2. Start BEGIN IMMEDIATE in a short SQLAlchemy transaction. Recheck the actor's active status and role under the write reservation.
3. Read the latest event ID and derive the current state from ordered events. Require the form's expected_version to equal the current ID.
4. Validate the requested transition and capacity. A rejected movement writes an audit row and commits no movement change.
5. For an accepted transition, insert the event with server-owned actor/time, insert its audit record, and commit both together.

SQLite serializes competing writers. A second request sees the first committed event and fails the stale-version check. The global version conservatively rejects forms even if another person's event caused the change; the user refreshes and tries again. This is an intentional classroom-scale usability tradeoff.

Authorization and sessions

Every protected page uses login and role checks. Administrator URLs and POST actions are guarded directly. There are no event edit/delete routes. Guests never receive person names or history. Admin changes cannot change the current admin's own role/status. Disabling or changing another account invalidates that user's previous sessions.

Flask-Login identifies the session, while a database session_version lets logout revoke copied old cookies. Logout signs out all sessions for that account. Cookies expire after 30 minutes; renewed cookies cannot revive a revoked version. Password reset/MFA and per-device session management are PLANNED, not present.

STRIDE threat model

Each category has a concrete attack scenario, affected component, impact and mitigation. Threats extend Gustavo's draft; Tampering corrects the draft's spelling and Denial of Service supplies the missing category. See the Phase I SEC and BL identifiers for requirement mapping.

T1 Spoofing

Component: Login and session cookie. Scenario: an attacker guesses an employee password or forges a cookie using a leaked signing secret. Impact: fake movements under another identity. Mitigation IMPLEMENTED: salted script hashes, generic login failures, same-cost dummy hash verification, a local 10-failure/15-minute source throttle, a secret generated from at least 32 random bytes in ignored .env, and cleared login session data. Mitigation PLANNED: HTTPS deployment, distributed-rate testing and optional MFA. Requirements: SEC-01, 03, 08, 09, 10.

T2 Tampering

Component: Event form, movement service and event ledger. Scenario: staff or an attacker submits a second room entry, wrong-room exit, forged actor/time or simultaneous conflicting events. Impact: impossible occupancy or false attribution. Mitigation IMPLEMENTED: server-side transition rules, strict field allow-lists, server actor/time, foreign keys, expected_version and BEGIN IMMEDIATE validation/write. History has no editing endpoint. Residual: a host/database owner can edit SQLite directly. Requirements: BL-01 through BL-10 and SEC-04.

T3 Repudiation

Component: GalleryEvents and AuditLogs. Scenario: an employee records a false event or an administrator disables a user, then denies the action. Impact: inability to investigate changes or assign responsibility. Mitigation IMPLEMENTED: session-derived actor, timestamp, action, target and result; accepted movements and their audit rows commit together; rejected movements are recorded; audit UI is admin-only with no edit/delete action. Mitigation PLANNED: external protected retention, backup verification and regular review. Logs are not cryptographically tamper-evident. Requirements: SEC-07 and NFR-06.

T4 Information Disclosure

Component: History, dashboard and errors. Scenario: a guest guesses a person ID or requests an admin URL to obtain movements, hashes or diagnostic details. Impact: privacy loss or information useful for further attacks. Mitigation IMPLEMENTED: aggregate-only guest dashboard; staff-only history; admin-only audit; generic errors; no hash field in templates; parameterized queries; no-store responses and autoescaped output. Mitigation PLANNED: HTTPS and host encryption before real use. Requirements: SEC-02, 05, 09, 11, 12.

STRIDE continued and security checklist

T5 Denial of Service

Component: Login hashing, request parsing, historical queries and SQLite writes. Scenario: repeated guesses consume hash CPU, large forms exhaust memory, or heavy event traffic grows storage and contends for database locks. Impact: slow or unavailable gallery operations. Mitigation IMPLEMENTED: 16 KiB body/form bounds, bounded form parts, history pagination at 25 rows, 10 failed attempts per source per 15 minutes, and a finite SQLite busy timeout. Mitigation PLANNED: load testing, proxy/server resource limits, reliable distributed throttling, disk monitoring and retention. The simple throttle can be exceeded by simultaneous or distributed attempts and can affect users sharing one IP. Requirements: SEC-10, NFR-01, 02, 06.

T6 Elevation of Privilege

Component: Role enforcement and user administration. Scenario: an employee types /admin/users or submits a forged role=admin field to a normal form. Impact: unauthorized account creation, privilege changes or audit disclosure. Mitigation IMPLEMENTED: database-loaded roles, role decorators on protected routes, actor role recheck before writes, strict field allow-lists, admin-only user endpoints and CSRF protection. Role or active-status changes invalidate the affected account's sessions. No self-promotion endpoint exists. Requirements: SEC-02, 03, 04, 06 and BL-10.

Implementation checklist

Control	Status and evidence
Passwords and secrets	IMPLEMENTED: models.py script methods; setup_local.py generates secrets; .gitignore and .dockerignore exclude private files.
Auth, roles and disabled accounts	IMPLEMENTED: application user_loader; routes.py guards; functional and access tests.
SQLi and XSS defenses	IMPLEMENTED: SQLAlchemy operations, Jinja autoescape and restrictive CSP; targeted injection/encoding tests.
CSRF and session flags	IMPLEMENTED: global Flask-WTF, tokens in every POST form, HttpOnly/Strict cookies; production Secure/HSTS configuration. HTTPS deployment PLANNED.
Movement integrity	IMPLEMENTED: services.py transactions, state rules and stale-form checks; concurrency/state tests.
Audit and safe errors	IMPLEMENTED: explicit action logging and generic error pages. Durable external logs/alerts PLANNED.
Availability and recovery	Request/page bounds IMPLEMENTED. Sustained load testing, protected backups, restore drill and retention PLANNED.

Changes from Gustavo's draft

The DFD now draws trust boundaries. SECRET_KEY is at least 32 random bytes, not 32 bits; the setup stores its hex representation in ignored .env. Room entry requires gallery presence and no existing room; gallery exit requires no current room. Own-only guest history was replaced by aggregate-only guest access. Account lockout, audit triggers and PersonState were simplified to source throttling, explicit audit writes and ledger reconstruction. Claims of deployment controls are labeled PLANNED.

Validation scope and references

Verification approach

Automated tests exercise authentication, the three roles, disabled accounts, CSRF, password hashing, logout revocation, malformed and forged requests, history filters, output encoding, all movement invariants, room capacity, stale replay and two-writer contention. A local browser smoke test demonstrates Browser -> Flask -> SQLite. Exact execution outcomes are recorded in `verification_results.md`; report statements must be read with that evidence.

Known limitations

State reconstruction scans the full event ledger; person selectors and the admin list are not designed for large datasets. SQLite supports one writer at a time. The source-based login throttle is a basic local safeguard, not a full DoS solution. Cookies are signed but not encrypted and hold no sensitive personal data. SQLite, `.env` and local credentials are plaintext files protected by host access; host encryption and deployment TLS remain planned.

There is no public signup, password-reset workflow, MFA, per-device logout, event correction workflow, event deletion, scheduled backup, retention job, alerting or cryptographic audit signing. These are not required Week 6 features. Docker files are provided as the specified repository structure requires, but container execution must be independently verified if no Docker runtime is available.

Future verification

Week 8: expand SQLi/XSS, IDOR, session, authentication, privilege and misconfiguration testing with authorized tools. Week 9: install on the partner's machine, confirm deployment instructions, and run the instructor's oracle when supplied. Week 10: attack only the instructor-assigned authorized target. Week 11: reproduce confirmed findings, fix root causes and add regression tests. These phases are not represented as complete.

References and provenance

F26_BIBIFI_Project.pdf, sections 1-3 and 5-6, pages 1-7: required architecture, roles, security, business logic, diagrams, STRIDE, weekly milestone and repository structure.

BIBIFI doc.docx by Gustavo Pepe: source draft for the DFD flow, five original STRIDE threats and security checklist. The original file remains unchanged.

Project_Phase1-1.pdf, pages 1-3: functional/non-functional requirements, CIA/accountability, risk and compliance considerations; Phase I PDF submission.

Flask security guidance: <https://flask.palletsprojects.com/en/stable/web-security/>

Flask-WTF CSRF protection: <https://flask-wtf.readthedocs.io/en/latest/csrf/>

See `phase1_requirements.pdf` for prioritized requirements and compliance references, `api.md` for endpoint contracts, `requirements_audit.md` for official-source traceability and `README.md` for reproducible setup.